

Note-set demo, part 2 (Lecture 02) — a concept of operations, operations with contracts, the specification as an invariant

Part 2 of the demonstration begun in Lecture 01. The repository starts as part 1 left it, plus a draft concept of operations and the process documents that govern operations. The class watches the concept of operations audited and amended, a lower-level document derived from it, three operations performed with conformance re-established after each, and a change of requirement that moves the specification, the verifier, and every note in one commit.

The caution from part 1 applies unchanged: rehearse; capture at every [fallback capture]; every segment has an *If it goes differently* note.

What the starter contains, and what the audit should find

`starter-102/` is the end of part 1 — `note-format-spec.md` 1.0.0, one note, `index.md`, `check_notes.py` for 1.0.0 — plus: `note-set-conops.md` at 0.1 (draft); the L02 `CLAUDE.md`, which names the concept of operations as the first governing document; the L02 `process/`, which adds `README.md` (the invariant), `development-rules.md` (DEV), and `conops-audit.md` (AUDCON); and `BACKLOG.md`.

The audit in Segment 2 should find at least these. Do not reveal them first.

#	Finding	Rule	Scripted ruling
1	§2 says the formatting of “markdown files in the note set” is governed by the format specification; specification 1.0.0 says <code>index.md</code> is not a note	AUD-3, between documents	amend §2: the index is not a note; only R6 applies to it
2	O1 supplies a title; nothing in the ConOps says what file is created, and R7 now settles it	AUD-4; AUDCON-3	amend §2: a note’s filename is derived from its title per R7
3	O2’s postcondition contains “preserving the user’s meaning” — a judgment clause with no stated verifier	AUDCON-3	no amendment; it is a judgment clause and the VER-2 table already names a human or agent verifier for it
4	§5 says reports cite rule identifiers of the format specification	AUDCON-5 (SHOULD)	no change: it describes what the user sees
5	§6 “every operation ends in a git commit” reads like a process rule (DEV-7)	AUDCON-2	no change: the user observes the commit; the rule that enforces it is DEV-7

Rulings 3–5 are recorded in the RPT-1 status column as *ruled: no change*, with the reason. That is the point of listing them: a finding is not always a change.

Before class — setup checklist

1. Create the demo repository from `starter-102/`:

```
rm -rf ~/note-set-demo
cp -R <path-to-this-demo>/starter-102 ~/note-set-demo
```

```
cd ~/note-set-demo
git init && git add -A
git commit -m "part 2 start: specification 1.0.0, conops 0.1, process"
```

(Continuing part 1's repository is fine too: copy in the added files and commit them under the same message.)

2. Confirm `python3 check_notes.py --all` exits 0.
3. Copy `completed/check_notes.py` (the 2.0.0 verifier) outside the repository as the Segment 5 fallback.
4. Open `sample-inputs/no-silver-bullet-content.md` and `sample-inputs/lehmans-laws-content.md` in a plain-text editor.
5. Start Claude Code in the repository; confirm the three always-on process files loaded. Two terminals, large font.
6. Rehearse once; capture; reset by repeating step 1.

Segment 1 — A new kind of document (about 6 minutes)

Do (shell): `ls`, then `cat note-set-conops.md`.

Say: this is a concept of operations: a specification of purpose. It says what the set is for (§1), what it consists of (§2), who acts (§3), which operations the agent performs and what each requires before and guarantees after (§4, O1–O3), that conformance is an invariant of the set (§5), and how change is managed (§6). The preconditions and postconditions are Meyer's contracts, applied to operations rather than to methods. **Version: 0.1 (draft):** it has not been audited.

Do (shell): `cat process/README.md; ls process/`.

Say: the invariant, in three clauses; and two new documents — DEV, the rules for how an operation begins and ends and who may change what, and AUDCON, the audit rules that apply only to a concept of operations.

Segment 2 — Audit the concept of operations (about 12 minutes)

Do: plan mode. Prompt:

Read `note-set-conops.md`, `note-format-spec.md`, and the process documents. Run the audits in `process/conops-audit.md` and `process/spec-audit.md` on the concept of operations. Check it against the format specification (AUD-3) and walk through each of O1, O2, and O3 (AUD-4). Report one gap list per RPT-1 and wait for my rulings.

Expected: a list containing the five findings above, usually more.

[fallback capture] — the list.

Do: rule per the table. For anything else: one sentence, or “defer it to `BACKLOG.md`” (DEV-6).

Do: prompt:

Propose the amendments to `note-set-conops.md` per RPT-3 and wait for my approval.

Do: approve. Leave plan mode; the agent edits, sets 1.0, writes the changelog entry, commits as `conops: 0.1 -> 1.0`.

Do (shell): `git diff HEAD~1 -- note-set-conops.md`.

Say: finding 1 was an inconsistency *between* two documents; the format specification had already moved in part 1, and the concept of operations had not followed. One intent, two documents, kept consistent by an audit that reads both. Findings 3–5 changed nothing, and the report says why; a ruling of “no change” is still a ruling.

If it goes differently: nudges — 1: “Read §2 against the scope statement of the format specification.” 2: “Walk through O1: what file appears?” 3: “Who decides whether meaning was preserved?”

Segment 3 — Derive the operations document (about 8 minutes)

Do: prompt:

Derive `note-set-operations.md` from the concept of operations and the format specification (DEV-5): for O1, O2, O3, and a check-conformance operation O4, give the input, the precondition, the steps, and the postcondition; state the conventions — the slug per R7; every operation ends with O4 and a commit named for it (DEV-7). Do not create any note. Audit the new document against both governing documents (AUD-3) and report per RPT-1; then commit as `operations: 1.0`.

Expected: the document written; the audit finds nothing, or small items ruled in one sentence each; a commit.

Do (shell): `cat note-set-operations.md`.

Say: three specifications now govern this repository, each an abstraction of something different: the concept of operations abstracts purpose; the format specification abstracts every conformant note; the operations document abstracts the agent’s behavior. The last is derived from the first two and, by DEV-5, can never contradict them — and it is binding on the agent that wrote it.

If it goes differently: if the agent creates a note anyway, remove it with O3 once Segment 4 begins — an early demonstration of remove.

Segment 4 — Operations, and the invariant (about 12 minutes)

Do: prompt:

Add a note titled “No Silver Bullet” with the initial content I am about to paste. Bring it into conformance with the format specification, preserving my meaning. End per DEV-7: run the gate, report per RPT-5, commit.

Paste `sample-inputs/no-silver-bullet-content.md`.

Expected: the pasted `##` sections under front-matter and the correct H1, indexed, `check_notes.py --all` exit 0, an RPT-5 completion note that names the O2 judgment clause as reviewed by the agent, a commit named for the operation.

Do: prompt:

Add a blank note titled “Design by Contract”, then remove the note titled “Design by Contract”. End each operation per DEV-7.

Expected: add, then remove; the file gone and the index entry gone; two commits.

Do (shell): `ls notes/; git log --oneline`.

Say: R6 held through every operation. Conformance is an invariant of the set, checked after every change, not a milestone reached once. The commit log names each operation: it is the realization of the process documents, and reading it is how the process is audited.

If it goes differently: if the agent objects to the add-then-remove pair, say why — O3’s precondition needs a note to exist — and ask once more.

Segment 5 — A new requirement: specification 2.0.0 (about 12 minutes)

Say: requirements change. “I want the index to tell me what each note says, not only that it exists.”

Do: prompt:

New requirement: every note begins with a **## Summary** section — one paragraph, the first section after the title — and each index entry shows the first sentence of that summary. This is a specification change. Propose per RPT-3: a new rule R8, an amended R6, version 2.0.0 with a changelog entry; the matching changes to `note-set-operations.md`, to `check_notes.py` and its docstring's version, to the VER-2 table in `process/verification.md`, to the two existing notes, and to `index.md`. Show me the amendments first and wait.

Do: approve.

Expected: the specification at 2.0.0 with a breaking-change entry; the verifier decides R8 and the new R6 entry form; the VER-2 table gains R7, R8's MUST clause as mechanical, and R8's SHOULD clause as judgment; the agent re-reads each note and writes its summary; the index is rebuilt; `--all` passes; one migration commit.

[`fallback capture`] — the two summaries.

Say: one requirement changed the specification, the verifier, two notes, the index, and a process table, in one commit, with conformance re-established at 2.0.0. VER-3 is the rule: after an amendment, everything it touches is re-checked against the new version — a verifier still at 1.0.0 would have reported both notes conformant, a false pass. The summaries are judgment work: R8's MUST half (a section exists, one paragraph) is decided by the program; its SHOULD half (the summary is accurate) only a human or an agent can decide, and the VER-2 table now says so. One question the change raises — must a blank note created by O1 carry a placeholder summary? — is recorded in `BACKLOG.md` (DEV-6), not guessed; check that the agent did so, and ask if not.

Optional, if ahead: prompt to add “Lehman’s Laws of Software Evolution” with `sample-inputs/lehmans-laws-content.m`

Expected: a note born conformant at 2.0.0, with its summary written unprompted, and the apostrophe in the title exercising R7.

If it goes differently: if the agent migrates before showing amendments, stop it once: “amendments first — RPT-3.” If a summary is weak, tighten R8's SHOULD clause by amendment and re-run the migration: the last segment can still change the specification.

Segment 6 — Close (about 4 minutes)

Say: walk the table.

Claim	Evidence from the two lectures
A specification is a continuously maintained document of intent	0.1 → 1.0.0 → 2.0.0 under git, each step a ruling with a changelog entry; a backlog for what is not yet decided
A specification is an abstraction of the realization	R1–R8 describe every conformant note and none in particular; the operations document describes the agent's behavior without being it
Conformance is an invariant while both sides change	the specification moved (part 1, Segment 2; part 2, Segment 5); the realization moved (part 1, Segment 3); a check after every operation
There are several specifications, and they must be consistent	three documents; finding 1 of Segment 2 was between two of them, and both now agree
Information flows both ways	the audit produced R7 and the R3 amendment before any note existed; writing the verifier fixed R2's front-matter to two scalar fields; a new requirement changed five artifacts at once

Say: the note in this set — Royce, 1970 — is the paper cited as the origin of single-pass development, and it argues against single-pass development. The first realization in the repository states the lecture's claim.

Recreating this part yourself (students)

Copy `starter-102/` outside the course repository, follow the checklist, and run Segments 2–5. Your gap list, your rulings on findings 3–5, and your summaries will differ from the script's. Compare your `note-set-conops.md` and `note-set-operations.md` with `completed/`.